

Plano do Projeto — Agente CLI Python

Runtime de Agentes + Ollama + Docker + VPS

Objetivo: construir uma plataforma de agentes em Python, executável por CLI, com memória, ferramentas, planejamento e modelos intercambiáveis, preparada para implantação em uma VPS Linux usando Docker.

Base documental: materiais fornecidos sobre Ollama como motor de ação, arquitetura do Agente CLI Python e implantação com VPS/Docker. As decisões abaixo preservam o que esses materiais sustentam e distinguem recomendações de arquitetura.

1. Visão do produto

O projeto não deve ser tratado como um chatbot isolado. A proposta é criar um **Agent Runtime**: um núcleo de software responsável por coordenar modelos, memória, ferramentas, contexto, planejamento, execução e permissões. Diferentes agentes especializados poderão utilizar o mesmo runtime.

Exemplo de evolução: um agente principal, um agente de código, um pesquisador e um revisor podem compartilhar o mesmo motor, enquanto cada agente define seu modelo, ferramentas, regras e objetivo.

Princípio arquitetural: **separar o runtime do modelo**. O Ollama fornece capacidades de inferência e tool calling; o runtime Python controla o ciclo de execução e as ferramentas.

2. Arquitetura proposta

```
INTERNET / CLI
↓
Agent Runtime — Python
  ■■■ Context Manager
  ■■■ Memory Manager
  ■■■ Planner / Executor
  ■■■ Tool Registry + Permissions
  ■■■ Model Provider
↓
Ollama
↓
Modelo local
```

Na infraestrutura, a aplicação Python e os serviços auxiliares podem ser isolados em containers Docker. Uma composição inicial pode conter Agent API/Runtime, Ollama, PostgreSQL e, posteriormente, Redis e um proxy HTTPS.

3. Papel do Ollama

O Ollama deve funcionar como **motor de decisão/inferência**, e não como executor irrestrito de ações. O fluxo de tool calling é: o runtime envia contexto e ferramentas disponíveis; o modelo retorna uma chamada de ferramenta; o backend valida e executa; o resultado retorna ao histórico; o modelo decide o próximo passo ou produz a resposta final.

Capacidades relevantes para o projeto incluem chat, tool calling, structured outputs, streaming, embeddings, vision, thinking e controle de contexto. O material também destaca a compatibilidade com uma interface OpenAI-compatible, útil para integrar SDKs e ferramentas existentes.

Nota: o material fornecido recomenda contexto alto para agentes; a capacidade real depende do modelo e dos recursos disponíveis na máquina.

4. Componentes do runtime

CLI: entrada do usuário e comandos de operação. Não deve conter a lógica principal do agente.

Agent Core: coordena contexto, memória, modelo, ferramentas, planejamento e resposta.

Memory Engine: separa memória de curto prazo, memória episódica, memória semântica e estado de trabalho. O histórico bruto não deve ser a única estratégia de memória.

Tool Registry: catálogo de ferramentas com schemas, validação, permissões, timeout e auditoria.

Planner/Executor: transforma objetivos em etapas, executa ações e valida resultados.

Model Provider: abstração que permite trocar Ollama por outro provedor sem reescrever o Agent Core.

5. Estrutura de projeto sugerida

```
agent-cli/  
  main.py  
  pyproject.toml  
  agent/ - core, planner, executor, context  
  memory/ - manager, short-term, long-term, storage  
  models/ - interface e providers  
  tools/ - registry, filesystem, shell, git, web  
  agents/ - configurações dos agentes especializados  
  data/ - memória, conhecimento e logs  
  tests/ - testes do agente, memória e ferramentas
```

6. Stack inicial

Camada	Tecnologia / decisão
Interface	Python CLI
Runtime	Python
Inferência	Ollama
Modelo	Modelo compatível com tools; Gemma 4 aparece no material como candidato multimodal/agentic
Memória / estado	PostgreSQL como opção inicial de persistência
Cache / filas	Redis — somente quando houver necessidade
Isolamento	Docker / Docker Compose
Infraestrutura	VPS Linux; Hostinger é uma opção
HTTPS / entrada	Nginx ou proxy equivalente, quando houver API externa

7. VPS + Docker: decisão de infraestrutura

A combinação VPS + Docker é adequada para hospedar o runtime, mas a escolha da VPS precisa ser feita **depois de definir o modelo e medir o desempenho necessário**. O ponto crítico é a inferência: CPU pode executar modelos, porém modelos maiores podem exigir recursos consideráveis; uma GPU compatível muda o cenário.

Portanto, não tratar 'ter Docker' como sinônimo de 'ter capacidade para rodar qualquer modelo'. Docker resolve isolamento e implantação; não resolve falta de CPU, RAM, VRAM ou largura de banda de memória.

Estratégia recomendada: começar com um modelo quantizado e carga pequena, medir latência/memória e somente depois dimensionar a VPS definitiva.

8. MVP — ordem de implementação

Sprint	Entrega	Critério de conclusão
1	CLI + Agent Core + provider Ollama	Enviar mensagem e receber resposta de forma estável
2	Tool Registry + 2 ou 3 tools seguras	Modelo chama ferramenta; runtime valida, executa e retorna resultado
3	Memória de sessão + persistência	Contexto relevante sobrevive entre execuções
4	Planner / Executor + structured outputs	Objetivo multi-etapa vira plano executável

Sprint	Entrega	Critério de conclusão
5	Docker Compose + observabilidade	Subida reproduzível, logs e métricas básicas
6	Testes e hardening	Timeouts, retries, permissões e casos de falha cobertos

9. Segurança e confiabilidade

O runtime deve tratar tool calling como uma solicitação não confiável até que seus argumentos sejam validados. Cada ferramenta deve ter schema, permissões e limites. Ferramentas com efeitos externos devem exigir controles mais fortes que ferramentas de leitura.

Itens mínimos: validação de argumentos, allowlist de ferramentas, timeout, retry controlado, logs de auditoria, isolamento de execução, limites de recursos e tratamento explícito de erros.

10. Memória e RAG

Separar **Memory** de **Knowledge**. Memory guarda estado e fatos relevantes sobre sessões/projetos. Knowledge contém documentos, código e outras fontes consultáveis. Embeddings podem sustentar busca semântica/RAG; isso não substitui o gerenciamento do estado da tarefa.

11. Voz e multimodalidade

A camada de voz deve ficar desacoplada do Agent Runtime: áudio → Speech-to-Text → Agent Runtime → Ollama/tools → resposta → Text-to-Speech. O material fornecido aponta evidência para entrada/transcrição de áudio, mas não estabelece um TTS nativo estável como componente principal da API; portanto, TTS deve ser tratado como serviço/camada separada até validação.

Vision pode ser adicionada quando houver necessidade de analisar screenshots, imagens ou documentos visuais, desde que o modelo escolhido suporte essa capacidade.

12. Evolução para multi-agent

```

Agent Runtime
■■■ Atlas – agente principal
■■■ Coder – código e engenharia
■■■ Researcher – pesquisa e síntese
■■■ Reviewer – validação e revisão

```

Os agentes compartilham runtime, memória e infraestrutura, mas podem receber diferentes prompts, ferramentas, modelos e permissões. O objetivo é adicionar agentes sem duplicar o núcleo.

13. Critérios de sucesso do MVP

1) instalação reproduzível; 2) CLI funcional; 3) chamadas de ferramentas previsíveis; 4) memória persistente funcionando; 5) logs suficientes para entender cada execução; 6) falhas de ferramenta não derrubam o agente; 7) modelo pode ser trocado através da camada de provider; 8) custos e consumo de recursos são mensuráveis.

14. Decisão final recomendada

Construir primeiro o Agent Runtime em Python, usando Ollama como provider local, Docker como camada de empacotamento e uma VPS Linux como infraestrutura de implantação. Não começar pelo multi-agent, voz ou automação irrestrita. Primeiro provar o loop fundamental: contexto → modelo → tool call → validação → execução → resultado → próxima decisão → resposta.

Depois que esse loop estiver sólido, memória, planner, voz, vision e múltiplos agentes entram como extensões do mesmo runtime.

Documento elaborado a partir dos materiais fornecidos nesta conversa. Pontos de infraestrutura e dimensionamento que dependem do plano específico da VPS ainda precisam ser validados antes da contratação.